



EAST TEXAS A&M
— UNIVERSITY —

CSCI 380.61E
WEB PROGRAMMING
AND INTERFACE DESIGN

OVERVIEW

- What is **AJAX** and **JSON**?
- How do we use AJAX?
- **Syntax** and formatting
- AJAX methods and properties
- **Examples**
- Questions and Discussion

REVISION QUESTION

What is the result?

```
(true + true) > 1 + false
```

- A. true
- B. false
- C. TypeError
- D. NaN

ANSWER

Answer: **A.** true

$(\text{true} + \text{true}) = 2, 1 + \text{false} = 1$

REVISION QUESTION

What is the output for the following JavaScript?

```
function foo( a )  
{  
  a = [5,6];  
}  
let b = [3,4];  
foo( b );  
console.log( b[1] );
```

- A. 5
- B. 6
- C. 3
- D. 4

ANSWER

Answer: **D.** 4

JSON

- **JSON = JavaScript Object Notation**
- JSON is a subsection of JavaScript (ECMA-262)
- Language **independent**
- Text-base (**human** readable)
- Light-weight
- Easy to use and parse

JSON IS **NOT**

- JSON is not a document format
- JSON is not a markup language
 - **JSON is not XML**
- JSON is not a general serialization format
 - No recursive /recurring structures
 - No invisible structures
 - No function

WHAT DO WE USE JSON OBJECTS FOR?

WHAT DO WE USE JSON OBJECTS FOR?

- JSON objects are used to **transfer data** between the server and the client
- JSON is a **simple**, common representation of data

VERSIONS AND ENCODING

- JSON has **no version** and no revision
- JSON is very **stable**
- Character Encoding is **Strictly** UNICODE

JSON DATA TYPES AND VALUES

- Strings
- Numbers
- Booleans
- Objects
- Arrays
- null, true, false

JSON OBJECTS

- Objects are *unordered* containers of key/value pairs
- Objects are wrapped in **{ }**
- Comma **,** separates key/value pairs
- Colon **:** separates keys and values
- Keys are strings
- Values are from JSON data types
- Also refereed to as (in other languages) :
 - struct, record, hashtable, associative array, ...

OBJECT

```
{  
  "name": "Jack",  
  "at"   : true,  
  "grade": "A",  
  "format": {  
    "type": "rect",  
    "width": 2220,  
    "framerate": 24 }  
}
```

JSON **ARRAY**

- Arrays are *ordered* sequences of values
- Values can be of different types
- Arrays are wrapped in **[]**
- Comma **,** separate values
- Implementation can start at index 0 or 1

JSON ARRAY EXAMPLE

```
{  
  "vals": [ "Bob", "Tom", 22 ]  
}
```

EXAMPLES

Object with two member

```
{  
  "course": "6.69",  
  "cname" : "aaa",  
}
```

Array with 3 elements

```
[ 'aa', 'bbb', 'cccc' ]
```

Object with an array as a value

```
{  
  "course": "abc",  
  "times": [ "2", "3", "4" ]  
}
```

EXAMPLES

Key-value pairs

```
var test = {  
  "fname": "Bob",  
  "lname": "Tom",  
  "age": 22  
};
```

Accessing the JSON objects:

```
document.writeln("FName: " + test.fname);  
document.writeln("LName: " + test.lname);  
document.writeln("Age : " + test.age );
```


EXAMPLES

Objects with **objects** and **array**

```
var a = {  
  "john":  
    {  
      "label": "jjj",  
      "data": [[ 22, 233], [22,33]]  
    },  
  "james":  
    {  
      "label": "bbbb",  
      "data": [[ 12, 33], [99,33]]  
    }  
}
```

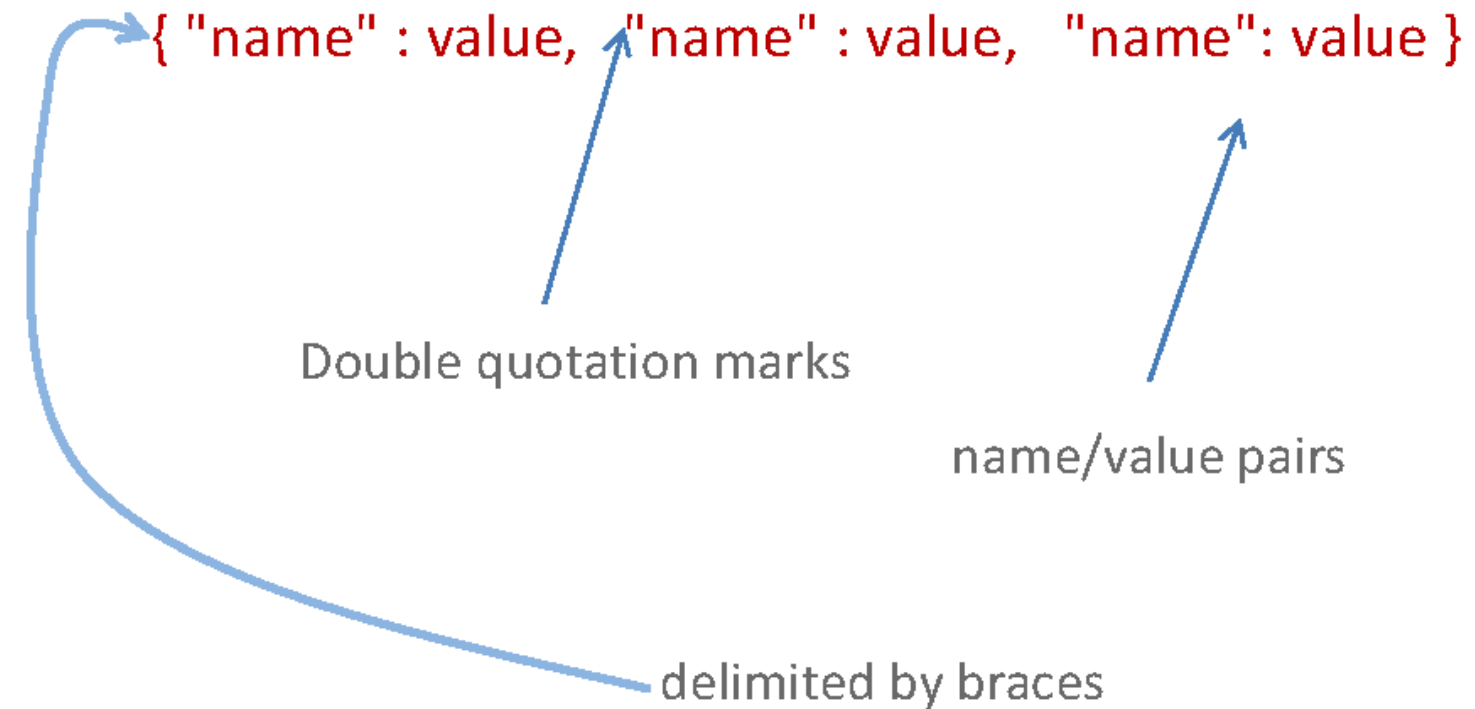
To access the data, we could do:

```
a.john.data[0][0]
```

Value: 22

{JSON} is a flexible yet strict JS object

JSON Object



{JSON} example

```
{
  "username": "jparker",
  "name"      : "John Parker",
  "address"   : {
    "address" : "444 Princes Street",
    "city"    : "Edinburgh"
  },
  "emails"    : [ "j.parker@gmail.com", "jp3@hw.ac.uk" ],
  "phone"     : null,
  "active"    : true,
  "age"       : 31
}
```

nested object

nested array

JAVASCRIPT AND JSON

`JSON.stringify()`

- Takes a Javascript object and produces a **JSON string** from it

`JSON.parse()`

- Takes a **JSON string** and parses it to a JavaScript object

EXAMPLE

Convert Javascript object into a string

```
var json = {  
  person: {  
    name: "bob",  
    age: 20  
  }  
};  
console.log( JSON.stringify( json ) );
```

Outputs:

```
{"person": {"name": "bob", "age": 20}}
```

EXAMPLE

JSON String

```
var json = '{"person" : {"age":20, "name": "bob"}}';
```

Convert to a Javascript object

```
var parsed = JSON.parse(json);
```

Test

```
console.log( parsed.person );  
console.log( parsed.person[ "age" ] );
```

JSON VS XML

Both have been used to communicate data

- JSON is valid JavaScript; better with JavaScript/Web than XML
- JSON less verbose and light weight
- XML can be used as a more general purpose markup
- XML usually specialized XML vocabulary is used
- JSON is much more universal in this regard

AJAX



AJAX

- AJAX is **not** a programming language
- AJAX is a technique for accessing web servers from a web page
- AJAX stands for **Asynchronous JavaScript And XML**

AJAX

- Technique for creating **interactive web pages**
- JavaScript and **XMLHttpRequest** to interchange and manipulate data with web server
- **Asynchronous** communication (i.e., no freezing up)

REVISION QUESTION

AJAX stands for?

- A. Abstract JavaScript Asynchronous XML
- B. Abstract JavaScript Adverse XML
- C. Asynchronous JavaScript Archive XML
- D. Asynchronous JavaScript And XML

ANSWER

Answer: **D.** Asynchronous JavaScript And XML

AJAX REQUEST

XMLHttpRequest is used to create and send HTTP requests

1. Create XMLHttpRequest:

```
let xhr = new XMLHttpRequest()
```

2. configure the request

```
xhr.open(method, URL, [async, user, password])
```

3. connect to server and send the request

```
xhr.send([body])
```

4. Listen to events for response **load , error , progress**

POST AND GET IN AJAX

- GET places arguments in **query string** (POST doesn't)
- GET call in AJAX has **size limitation** on the amount of data that can be passed

POST AND GET IN AJAX

General Principle

- GET method: it is used to **retrieve data** to display in the page and data is not expected to be changed on the server
- POST method: to update information on the server (**send**)

AJAX METHODS

- **open**("method", "URL"[,async[, "userName"[, "password"]]]):
 - Initializes a request.
- **send**([body]): Connect to server and Sends the request.
 - If the request is asynchronous(which is default), this method return as soon as the request is sent
 - If the request is synchronous, this method doesn't return until the response has arrived

AJAX CALLBACKS

- **onload**

```
xhr.onload = function() { ...
```

- **onerror**

```
xhr.onerror = function() {  
  alert("Request failed");  
};
```

- **onprogress**

```
xhr.onprogress = function(event) {
```

- `addEventListener(event, Callback)`
event: "load", "error", ...

AJAX RESPONSE

Receive the result in the following xhr properties:

- **status** - HTTP status code: 200, 404, 403, ...
- **statusText** - HTTP status message: OK, Not Found, Forbidden, ...
- **response / responseText / responseXML ...** - the response body

```
xhr.onload = function() {  
    if (xhr.status != 200) {  
        alert(`Error ${xhr.status}: ${xhr.statusText}`);  
    } else {  
        alert(xhr.response);  
    }  
};
```

- **responseType** - enumerated string value: arrayBuffer, json, document, text

EXAMPLE

```
<!DOCTYPE html>
<head><title>AJAX Test</title>
<script type="text/javascript">
    var xhr = new XMLHttpRequest();
    xhr.open("GET", "hello.txt", true);
    xhr.onload = ajaxCallback;
    xhr.send();
    function ajaxCallback(event){
        alert( "Your file contains the text: " + event.target.responseText );
    }
</script>
</head>
<body></body></html>
```

Note hello.txt could be changed to '.php' script (e.g., hello.php) instead that produces **dynamic content/information**

AJAX OTHER METHODS

- **setRequestHeader("label", "value")**: Sets the value of the HTTP request header
- **getAllResponseHeaders()**: Returns string with all header information
- **getResponseHeader("headerLabel")**: Returns the text of a specified header

AJAX OTHER METHODS/PROPERTIES

- **abort()** : Aborts request
- **onreadystatechange** - property contains the event handler when the readyState attribute changes

```
xhr = new XMLHttpRequest();  
xhr.onreadystatechange = function() {  
    if (xhr.readyState == 4 && xhr.status == 200)  
        alert(xhr.responseText);  
}  
xhr.open("GET", "hello.txt", true);  
xhr.send();
```

- **timeout** - (in ms)

```
xhr.timeout = 10000;  
xhr.ontimeout = function ....
```

JSON IN AJAX

- XMLHttpRequest
 - Obtain responseText
 - Parse the responseText

```
responseData = eval( '(' + responseText + ')' );
```

or

```
responseData = responseText.parseJSON( );
```

or

```
responseData = JSON.parse(responseText);
```

SECURITY

Is safer to use **eval** with XMLHttpRequest?

- JSON data comes from the same server that vended the page
- An `eval` of the data is no less secure than the original html
- If in doubt, use **string.parseJSON** or **JSON.parse** instead of `eval`

EXAMPLE JSON

```
<div id="demo"></div>

<script>
function getDoc() {
  const xhttp = new XMLHttpRequest();
  xhttp.onload = function() {
    document.getElementById("demo").innerHTML =
      this.responseText;
  }
  xhttp.open("GET", "https://f28wp.github.io/index.html");
  xhttp.send();
}
</script>
<button type="button" onclick="getDoc()">Get Content</button>
```

- Notice in this example grabs 'raw' html (could be json, txt or other data type)

FETCH

- **Fetch** is an interface for making AJAX calls in JavaScript ([Link](#)).
- Calling fetch returns a promise, with a Response object.

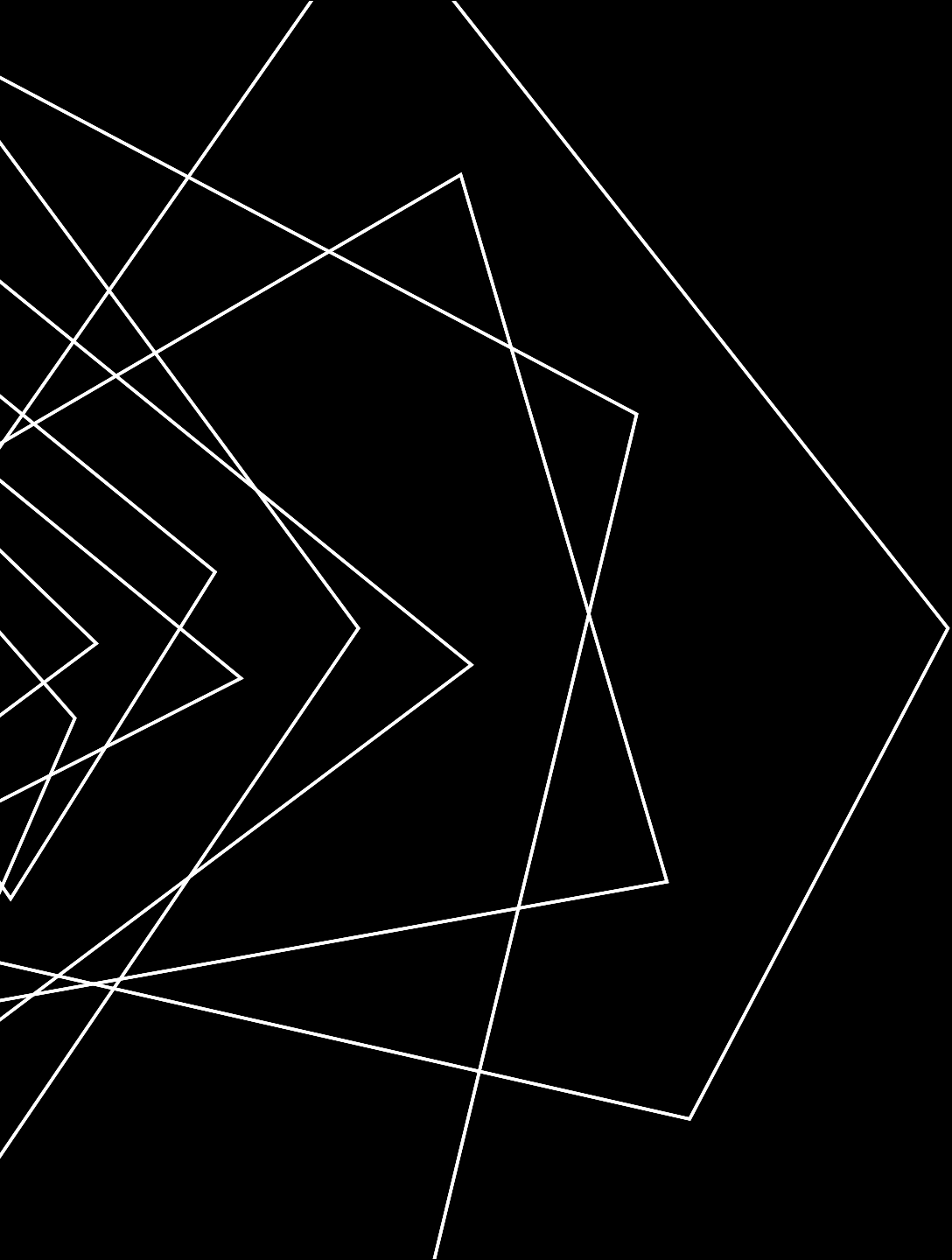
Example

```
const url = 'https://api.github.com/users/f28wp/repos';

fetch(url)
  .then(response => response.json())
  .then(repos => {
    const reposList = repos.map(repo => repo.name);
    console.log(reposList);
  })
  .catch(err => console.log(err))
```

SUMMARY

- Understand **AJAX** and **JSON**
- Formatting and Syntax
- Read/write JSON **structure**
- Benefits of AJAX
- Examples



THANK YOU

Yau Ka Cheung
Yauka.Cheung@etamu.edu
ACB1 Room 335